

SDG DMS — Handover & Context Document

For Claude Code in VSCode

Trinergy Digital | June 2026

1. Project Overview

This is a Document Management System (DMS) built for SD Guthrie (SDG), a client of Trinergy Digital. The developer is Clarence, a Junior Digital Developer at Trinergy.

The system allows employees to upload documents into a SharePoint staging area, tag them with metadata from the Term Store, and have approvers move approved documents to a final document library.

2. SharePoint Environment

Sandbox site: dcidigitalcom.sharepoint.com/sites/SPFX-Sandbox-Testing-Ground

Staging Library — Columns Created

- Department — Managed Metadata, linked to Department term set
- Document Type — Managed Metadata, linked to Document Type term set
- Year/Period — Managed Metadata, linked to Year/Period term set
- Confidentiality Level — Managed Metadata, linked to Confidentiality Level term set
- Project Name — Managed Metadata, linked to Project Name term set
- Vendor — Managed Metadata, linked to Vendor term set
- Document Date — Date column
- Approval Status — column for approval workflow

Term Store — DMS Metadata Group

Term Set IDs (sandbox tenant — dcidigitalcom.sharepoint.com):

Confidentiality Level: 032534ab-9285-4b42-98c6-5c7b0df1f066

Department: eaba82e5-3e5f-4719-9a76-091f034ad407

Document Type: 0540e66e-7cb3-47ac-b0ef-4e3069387394

Vendor: cb3c0ab7-a959-4200-9b7b-d1e13397d240

Year/Period: f7c578a1-e0e5-42ff-9e0c-d748cba42ede

Project Name: e1163337-93bb-4b6e-847e-346f51cc6806

Term Values (Dummy Data in Sandbox)

Confidentiality Level: Confidential, Internal, Public

Department: Finance, HR, IT

Document Type: Contract, Invoice, Policy, Report

Vendor: (no dummy values visible — check Term Store)

Year/Period: (no dummy values visible — check Term Store)

Project Name: Child terms nested under each Department term (Finance > projects, HR > projects, IT > projects)

3. Power Automate Flows

All flows built at make.powerautomate.com under the sandbox environment.

Flow 1: GetTermSetValues

Purpose: Reusable flow — fetches dropdown values from any Term Set given its ID.

- Step 1 — Trigger: When Power Apps calls a flow (V2). Input: Text called "TermSetId"
- Step 2 — Send an HTTP request to SharePoint:
Method: GET
Uri: `_api/v2.1/termStore/sets/{triggerBody()['text']}/children`
Header: Accept = application/json
- Step 3 — Select (reshapes the raw response):
From: `body('Send_an_HTTP_request_to_SharePoint')['value']`
Map Key: Id Value (expression): `item()['id']`
Map Key: value Value (expression): `item()['labels'][0]['name']`
- Step 4 — Respond to a Power App or flow:
Output name: terms
Output value (expression): `outputs('Select')`

Key gotchas:

- Use "Respond to a Power App or flow" — NOT the HTTP "Response" action
- Select step must exist before Respond step
- In Respond step use `outputs('Select')` with no `string()` wrapper
- After any flow edit, remove and re-add the flow in Power Apps to refresh schema

Flow 2: DepartmentProjectList

Purpose: Fetches project child terms under a specific department term.

- Step 1 — Trigger: When Power Apps calls a flow (V2). Input: Text called "DepartmentTermId"
- Step 2 — Send an HTTP request to SharePoint:

Method: GET

Uri: `_api/v2.1/termStore/terms/@{triggerBody()['text']}/children`

Header: Accept = application/json

- Step 3 — Select (same mapping as GetTermSetValues)
- Step 4 — Respond to a Power App or flow with terms output

Flow 3: UploadToStaging

Purpose: Receives a file + metadata and saves it to the Staging library.

Trigger inputs:

- FileName — Text
- FileContent — File (must be File type, NOT Text)
- Department — Text
- DocumentType — Text
- YearPeriod — Text
- DocumentDate — Text
- ProjectName — Text
- ConfidentialityLevel — Text
- Vendor — Text

Steps:

- Step 1 — Trigger: When Power Apps calls a flow (V2) with inputs above
- Step 2 — SharePoint: Create file
Site Address: `https://dcidigitalcom.sharepoint.com/sites/SPFX-Sandbox-Testing-Ground`
Folder Path: `/Staging`
File Name: `FileName` (from trigger)
File Content: `FileContent` (from trigger)
- Step 3 — SharePoint: Update file properties
Site Address: same as above
Library Name: `Staging`
Id: `ItemId` (dynamic content from Create file step)
Then map each metadata field to its matching trigger input

NOTE: FileContent MUST be File type in the V2 trigger — not Text. This is what allows binary file data to pass through.

4. Requirement Change — Now Using SPFx

The team has instructed the upload form must be built in SPFx instead of Power Apps. The Power Apps canvas app ("DMS Upload Form") is fully abandoned.

New Simplified Requirements for Upload Form

- Text input — user types the file name manually
- File picker — user selects one file only
- Dropdown — Department (loaded from Term Store)
- Upload button — submits file + metadata to Staging library

Keep it simple. One file, one filename input, one department dropdown, one button. Nothing else for now.

5. SPFx Project Details

Project folder: C:\laragon\www\Work\Projects\sd-gatrie

Node version: 22 (switch via: nvm use 22)

SharePoint tenant: dcidigitalcom.sharepoint.com

Workbench URL: https://dcidigitalcom.sharepoint.com/_layouts/workbench.aspx?debugManifestsFile=https://localhost:4321/temp/build/manifests.js&debug=true&noredir=true

Start dev server: npm run start

Build: npm run build

IMPORTANT: Uses newer SPFx toolchain (Heft, not Gulp). Do NOT use gulp serve.

6. Key Decisions & Rules

- Term Store is REQUIRED for all metadata dropdowns — do NOT use SharePoint Choice columns
 - Department dropdown must load from Term Store dynamically — not hardcoded
 - Only ONE file upload per submission
 - User manually types the file name in a separate text input
 - Power Apps work is fully abandoned — do not continue it
 - Do not suggest alternative approaches to Term Store unless strictly necessary
 - Project Name term set exists but is NOT used in the new simplified form
-

7. What To Build Next (SPFx)

Build a simple React-based SPFx web part in the sd-gatrie project with:

- Text input: File Name
- File picker: Select File (one file only)
- Dropdown: Department (loaded from Term Store using REST API or GetTermSetValues flow)

- Button: Upload — calls UploadToStaging flow with file + filename + department

Keep the code simple — no over-engineering.